

Owner Registration — Current Mobile Logic (Backend Handover)

Oct 2, 2026 · @ahmed nasr

Purpose

Today the mobile app runs the whole owner registration itself: it builds raw CRM (Dataverse OData) queries, makes every decision, and verifies the OTP on the phone. This document records that logic exactly as it runs today, so the backend team can move it into the API layer.

How every call travels today. The app never calls CRM directly. It encrypts a JSON envelope { Endpoint, Method, Body } and POSTs it to the proxy `CommunityAppProxy/HandleRequest`. The proxy forwards it to the CRM Web API (`.../api/data/v9.1/`) and returns the encrypted response. The proxy has no business rules: whatever OData URL the app builds is executed as-is.

Scope. Owner registration only (Join Palm Hills → Owner → Create account), plus the login path that sends an unfinished owner back into registration. Tenant, family member and invited-user flows are separate documents.

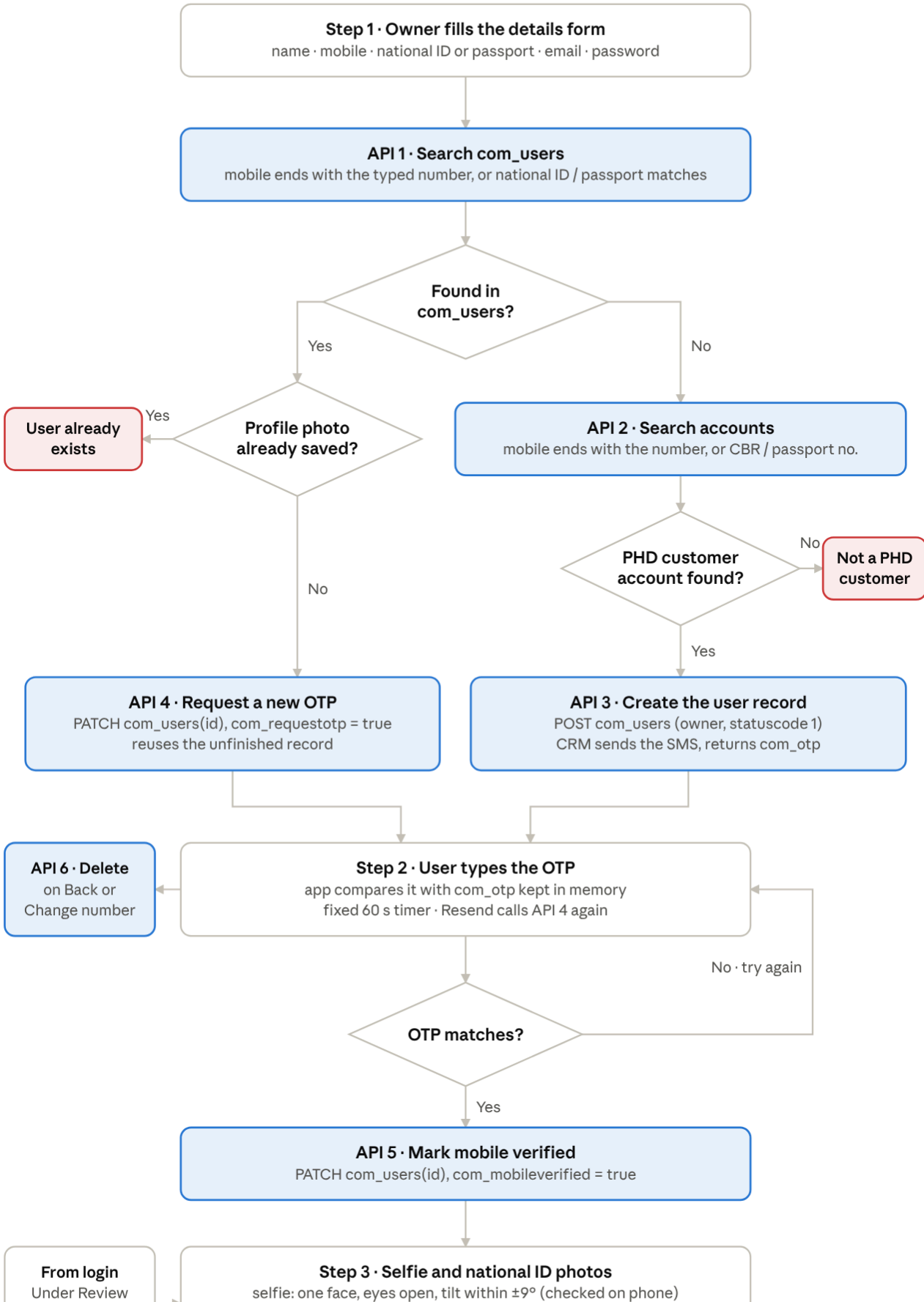
Source: `lib/screens/create_account/controller/create_controller.dart`,
`lib/repository/registration_repository.dart`,
`lib/repository/login_repository.dart`,
`lib/screens/login/controller/login_controller.dart`.

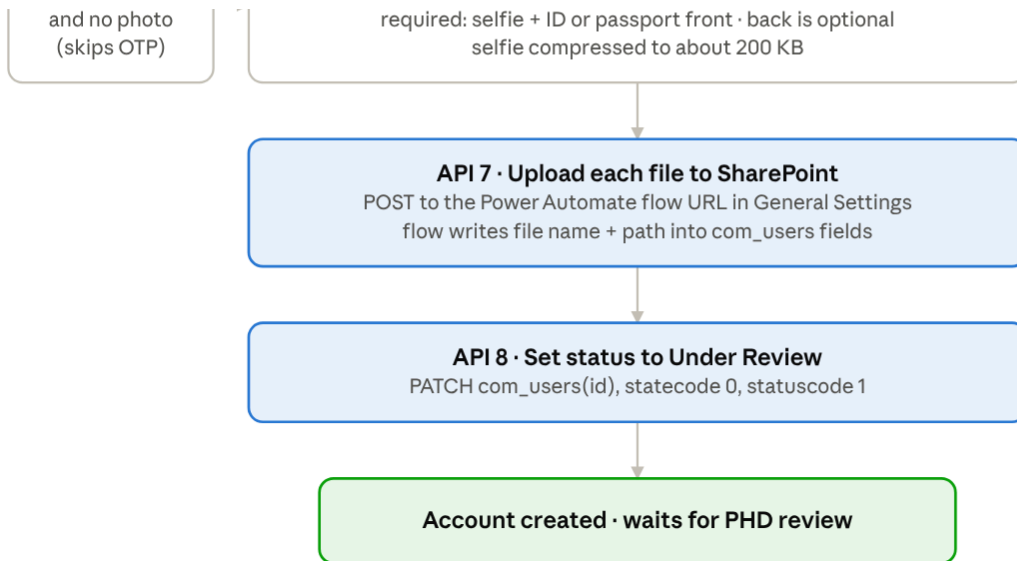
Current flow

Owner sign-up: two lookups pick the path, then OTP and photos

Numbers match the API sections below. Every call goes through the CommunityApp proxy.

○ App screen or app-only logic ○ API call through the proxy ○ Flow stops ○ Finished





owner registration as the app runs it today · 3 steps, 4 decisions, 8 API calls

Two lookups (API 1, API 2) decide whether the owner is new, unfinished or blocked; every path that continues goes through the phone-side OTP check and the photo upload before the record is set to Under Review.

Step by step: each API call and its business rule

The app makes up to 8 different calls. Numbers match the chart above. `{mobile}` is the number typed without the country code; `{dial}` is the selected country code (default 20).

Step 1 · Owner fills the details form (app only)

Fields: name (letters and spaces only), mobile + country code, national ID **or** passport (toggle), email, password. Validation on the phone:

- All fields required. National ID must be exactly 14 characters (no length rule for passport).
- Email must match `^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$`.
- Password must contain a digit, a lower-case letter, an upper-case letter and a symbol: `(?=.*\d)(?=.*[a-z])(?=.*[A-Z])(?=.*\W)`.

On "Submit and continue" the app runs API 1.

API 1 · Search existing app users (com_users)

Call	GET com_users
------	---------------

Query	<code>\$filter=(endswith(com_mobilenumber, '{mobile}')) or com_nationalid eq '{id}' — or com_passportnumber eq '{id}'</code> when passport is selected
Response read	<code>value[]</code> : first record's <code>com_userid</code> , <code>com_profilepicturefilepath</code>
Rule	A match on mobile or ID counts as "user exists". Only the first record is used.

Decision on the result:

- **No record** → go to API 2.
- **Record found**, `com_profilepicturefilepath` has a value → registration is complete. Stop with "This user already exists".
- **Record found**, `com_profilepicturefilepath` is empty → the user started before and never finished. Show "User exists but with incomplete profile, please complete it", then call API 4 on that existing record and go to the OTP step. The name, email and password just typed are **not** saved to that record.

API 2 · Check the owner is a PHD customer (accounts)

Call	GET accounts
Query	<code>\$filter=(endswith(new_mobilenumber, '{mobile}')) or new_cbrnumber eq '{id}' — or blser_passportno eq '{id}'</code> for passport
Response read	only the count of <code>value[]</code>
Rule	At least one CRM account must match by mobile or ID. Zero matches → stop with "User does not have an account in Palm Hills". No link to the account is stored.

API 3 · Create the user record

Call	POST com_users
------	----------------

Body	<code>com_name</code> (lower-cased), <code>com_mobilenumber</code> = + {dial}{mobile}, <code>com_nationalid</code> or <code>com_passportnumber</code> , <code>com_email</code> , <code>com_password</code> (plain text), <code>com_birthdate</code> (if set), <code>com_registered</code> : 1 (Owner), <code>statuscode</code> : 1 (Under Review), <code>com_requestotp</code> : true, <code>com_appnotificationtoken</code> (Firebase token, if any), <code>com_otpexpirydate</code> = now + 120 s
Response read	the full record, including <code>com_userid</code> , <code>com_otp</code> , <code>com_otpexpirydate</code>
Rule	<code>com_requestotp</code> : true makes CRM generate the OTP and send the SMS. The generated <code>com_otp</code> comes back to the phone in the response.

API 4 · Request a new OTP

Call	<code>PATCH com_users({userId})</code>
Body	<code>{ "com_requestotp": true }</code>
Response read	<code>com_otp</code> , <code>com_otpexpirydate</code>
Used when	(a) the existing-but-incomplete user from API 1, (b) the "Resend code" link on the OTP screen.

Step 2 · OTP check (app only)

- The app keeps `com_otp` from API 3 / API 4 in memory and compares it with what the user types. Match → API 5. No match → "OTP not correct", user retries.
- The countdown is hard-coded to 60 s. `com_otpexpirydate` is ignored because the backend value is about 6 hours off. When the timer hits 0 the Verify button is disabled and only "Resend" (API 4) works.
- **Back** or **Change number** on this screen calls API 6 and deletes the record.

API 5 · Mark the mobile as verified

Call	<code>PATCH com_users({userId})?</code> <code>\$select=com_mobilenumber</code>
------	---

Body	<code>{ "com_mobileverified": true }</code>
Rule	Sent only after the phone-side OTP match. The server never checks the OTP itself.

API 6 · Delete the user record

Call	<code>DELETE com_users({userId})</code>
Used when	Back or Change number on the OTP screen. Also deletes a pre-existing record that came through the "incomplete profile" path.

Step 3 · Identity documents (app only)

- **Selfie:** taken in-app and checked on the phone with Google ML Kit: exactly one face, both eyes open (≥ 0.99), head tilt within $\pm 9^\circ$ on all axes, face not touching the image edge, eye distance ≥ 60 px. Then compressed to about 200 KB.
- **National ID front** (or passport page): required. **Back:** optional for owners.
- Owner rule: selfie + front image required, otherwise "Please pick the required attachments".

API 7 · Upload each file to SharePoint

One call per file, all in parallel. Not a CRM call: the target is a Power Automate flow whose URL comes from General Settings (`sharepointattachmentcreationendpointurl`); the proxy is told `EnableAuthorization: false`.

Call	<code>POST</code> <code>{sharepointattachmentcreationendpointurl}</code>
Body	<code>attachmentid = userId, attachmentname ,</code> <code>attachmentbody (base64), siteaddress and</code> <code>folderpath (userattachmentsfolderpath) from</code> General Settings, <code>entityname: "com_users",</code> <code>filenamefield , filepathfield , oldfilepath:</code> <code>""</code>

Files	selfie.jpg → <code>com_profilepicturefilename / com_profilepicturefilepath ; front.png</code> or passport.png → <code>com_nationalidfrontname / com_nationalidfrontpath ; back.png</code> → <code>com_nationalidbackname / com_nationalidbackpath</code>
Rule	The flow saves the file and writes its name and path into the two CRM fields. <code>com_profilepicturefilepath</code> being filled is what later marks the registration as "complete" (API 1, login). Upload results are not checked by the app.

API 8 · Set status to Under Review

Call	<code>PATCH com_users({userId})</code>
Body	<code>{ "statecode": 0, "statuscode": 1 }</code>
Rule	Runs after the uploads, then the app shows "Account created". The record already had <code>statuscode: 1</code> from API 3, so for a new owner this changes nothing. PHD staff then approve (<code>181410000</code>) or reject (<code>181410001</code>) in CRM.

Side entry · Resume from login

Login queries `com_users` with `endswith(com_mobilenumber, '{mobile}')` and `com_password eq '{password}'` and then compares the password again on the phone. If `statuscode = 1` (Under Review) **and** `com_profilepicturefilepath` is empty, the app offers to finish registration. For an owner (`com_registered as = 1`) it jumps straight to Step 3 (documents), skipping the OTP even if `com_mobileverified` is false. If the account was rejected (`181410001`) with `com_allowresubmission = true`, it opens the same Step 3.

Logic the backend should own

Every decision below is made on the phone today. The OTP and password checks are the most urgent: the secret travels to the device and is compared there.

#	Decided on the phone today	Problem	Move to the API
1	OTP compare (<code>com_otp</code> returned in API 3 / API 4)	The OTP is visible to anyone who reads the response, so verification can be skipped	Server generates, stores and verifies the OTP; never return it
2	Mark mobile verified (API 5)	The app can set <code>com_mobileverified: true</code> without any OTP	Set only by the server after a correct OTP
3	OTP expiry and attempts	Expiry ignored (backend date ~6 h off), timer fixed at 60 s, no attempt limit	Server-side expiry and attempt limit; return remaining seconds
4	"User exists" check (API 1)	<code>endswith</code> on the mobile without country code can match another user's number	One check endpoint that matches the full <code>+{dial}{mobile}</code> or ID
5	"Complete or not" = <code>com_profilepicturefilepath</code> empty	Profile completeness depends on one file path	Explicit registration state on the record
6	PHD customer check (API 2)	Skipped for the "incomplete profile" path; account not linked to the user	Always check, and link the matched account
7	Record built by the app (API 3)	App sets <code>statuscode</code> , <code>com_registered</code> as , OTP expiry; password stored as plain text	Server sets status and role; hash the password
8	Delete on Back (API 6)	Can delete a pre-existing record from the incomplete path; any record id can be deleted	No delete from the app; server expires unfinished sign-ups
9	File uploads (API 7)	Upload results not checked; status set to Under Review even if a file failed	Server confirms all required files before moving to Under Review

#	Decided on the phone today	Problem	Move to the API
10	Under Review (API 8)	Already set at creation, so the final step adds nothing	Server moves the record to Under Review only when all steps pass
11	Login resume	Owner skips OTP when resuming, even if the mobile was never verified; password compared on the phone	Server returns the next required step
12	Proxy	Forwards any OData URL the app builds	Expose business endpoints only

Open question: should the selfie face checks (one face, eyes open, tilt $\pm 9^\circ$) stay on the phone for instant feedback, with a server re-check?yes

Backend build guide

The goal: the app sends only what the user typed or picked, and the API decides every next step. Endpoint names below are a proposal; keep the business rules even if the names change.

Rules for whoever builds it

- Never return a secret.** No response may contain `com_otp`, `com_password` or any other user's data. Every CRM read uses `$select` with only the columns the rule needs.
- The server owns state.** The app never sends `statuscode`, `statecode`, `com_registeredas`, `com_mobileverified`, `com_otpxpirydate` or a record id it wants to change. The server sets them.
- Escape every OData value.** Today the typed values are pasted into `$filter` strings. Double single quotes (`'` $\rightarrow$ `''`) or use typed query parameters, and validate the format first.
- Match mobiles exactly.** Normalise to E.164 (`+{dial}{digits}` , leading zeros stripped) and compare with `eq`, not `endswith`.
- Keep the same checks as the app.** Same validation rules as Step 1, so a request that passes the app also passes the API.
- One error code per outcome.** Return a stable `code` (see Reference) so the app can show today's message without knowing the rule.

7. **Registration token, not record id.** After the start call, return an opaque, short-lived `registrationToken` bound to the `com_userid`. Later calls use it; the app never sees or sends the CRM id.
8. **Keep CRM as the source of truth.** Same entities and fields (`com_users`, `accounts`), same SharePoint flow, same CRM plugin that sends the SMS when `com_requestotp` = true.

Proposed endpoints

Endpoint	Replaces	Does
POST <code>/registration/owner/start</code>	API 1, 2, 3, 4 (first OTP)	Validates, looks up the user and the PHD account, creates or reuses the record, triggers the OTP. Returns <code>registrationToken</code> , <code>otpExpiresInSeconds</code> , <code>nextStep: "verify-otp"</code>
POST <code>/registration/otp/resend</code>	API 4 (Resend)	Triggers a new OTP if the cooldown has passed. Returns <code>otpExpiresInSeconds</code>
POST <code>/registration/otp/verify</code>	Step 2 compare + API 5	Checks OTP, expiry and attempts on the server, sets <code>com_mobileverified</code> . Returns <code>nextStep: "upload-documents"</code>
POST <code>/registration/documents</code> (multipart)	API 7 + API 8	Requires a verified mobile. Uploads each file through the SharePoint flow, checks each result, then sets Under Review. Returns <code>nextStep: "under-review"</code>
POST <code>/registration/cancel</code>	API 6	Removes only a record created by this registration and not yet verified. Never deletes a pre-existing record
Login response gains <code>nextStep</code>	Side entry	<code>verify-otp</code> , <code>upload-documents</code> , <code>under-review</code> , <code>rejected-can-resubmit</code> , <code>rejected</code> , <code>deactivated</code> or <code>active</code> , so the app no longer reads <code>statuscode</code> or the photo path

Per query: why it moves and what the backend must do

API 1 · Search com_users

Why it moves

- No `$select`: the response holds the full `com_users` row of whoever matches, including `com_password` and `com_otp`. Anyone can look up any mobile or national ID and read that user's password.
- `endswith(com_mobilenumber, '{mobile}')` ignores the country code, so `1001234567` typed with `+971` still matches an Egyptian `+201001234567`.
- Typed values go straight into `$filter` (OData injection; a `'` in a passport number breaks the query).
- "Complete" is guessed on the phone from `com_profilepicturefilepath`.

Backend must (inside `/registration/owner/start`)

- Normalise mobile to E.164, then query `com_users` with `com_mobilenumber eq '{e164}'` or `com_nationalid / com_passportnumber eq '{id}'`, selecting only `com_userid`, `com_mobileverified`, `com_profilepicturefilepath`, `statuscode`, `statecode`, `com_registeredas`.
- More than one match (mobile hits one user, ID hits another) → return `USER_EXISTS`; do not pick the first row.
- Complete user → `USER_EXISTS`. Unfinished user → continue with that record (see Open questions on whether to overwrite its data).

Done when: no response carries `com_password` or `com_otp`; a `+971` number never matches a `+20` number; a passport value containing `'` returns a validation error, not a 500.

API 2 · Search accounts (PHD customer check)

Why it moves

- It is the core business rule "only PHD customers may register as owners", and today the app can skip it by calling API 3 directly.
- It is skipped entirely for the unfinished-user path.
- Same `endswith` and injection problems as API 1. The matched account is not linked, so staff reviewing the request can't see which account qualified it.

Backend must

- Query `accounts` with `$select=accountid` and `new_mobilenumber eq '{e164}'` (check the stored format first) or `new_cbrnumber / blser_passportno eq '{id}'`.

- Run it for every owner start, including unfinished users.
- No match → `NOT_PHD_CUSTOMER`. Match → keep the `accountid` and link it to the user if a lookup field exists (open question).

Done when: a mobile/ID with no account can never reach the OTP step by any path.

API 3 · Create the user record

Why it moves

- The app decides `statuscode`, `com_registered`, `com_otpexpirydate` and `com_requestotp`. Any client could register itself as another role or another status.
- `com_password` is stored as plain text.
- The response returns `com_otp` to the phone.
- The record is created as Under Review (`statuscode` 1) before the mobile is verified or any document exists.

Backend must

- Create `com_users` with `com_name`, `com_mobilenum` (E.164), `com_nationalid` or `com_passportnumber`, `com_email`, `com_password` (hashed — see Open questions), `com_appnotificationtoken`, and server-set `com_registered`: 1, `com_requestotp`: true.
- Set the status that means "started, not submitted" (see Open questions); not Under Review.
- Create with `Prefer: return=minimal` or `$select=com_userid` so the OTP never leaves the server.

Done when: the start response contains only `registrationToken`, `otpExpiresInSeconds`, `nextStep`; a new record is not visible as Under Review in CRM.

API 4 · Request a new OTP

Why it moves

- It PATCHes any record id the app sends: anyone can trigger SMS to any user, without limit.
- It returns `com_otp` in the response.

Backend must (`/registration/otp/resend`)

- Accept only `registrationToken`. Enforce a cooldown (today the app waits 60 s) and a max sends per registration and per mobile per hour.
- PATCH `com_requestotp: true` with `Prefer: return=minimal`.

Done when: a second resend inside the cooldown returns `OTP_RESEND_TOO_SOON`; no response contains the OTP.

Step 2 compare + API 5 · Verify OTP and mark the mobile verified

Why it moves

- The OTP is compared on the phone, then the app itself PATCHes `com_mobileverified: true`. Verification can be skipped by sending API 5 alone.
- No attempt limit. Expiry is ignored because `com_otpexpirydate` is about 6 hours off (a UTC vs local-time mismatch).

Backend must (`/registration/otp/verify`)

- Read `com_otp` and `com_otpexpirydate` server-side; compare in constant time; treat the date as UTC after fixing the offset at its source.
- Count failed attempts; lock after the limit (`OTP_ATTEMPTS_EXCEEDED`). Expired → `OTP_EXPIRED`. Wrong → `OTP_INVALID` with attempts left.
- On success: set `com_mobileverified: true`, clear `com_otp`, return `nextStep: "upload-documents"`.

Done when: `com_mobileverified` can only become true through this endpoint; an OTP works once.

API 6 · Delete the user record

Why it moves

- `DELETE com_users({id})` with any id: any client can delete any user.
- In the unfinished-user path, pressing Back deletes a record that existed before this session.

Backend must (`/registration/cancel`)

- Only delete when the token's record was created by this registration and the mobile is not yet verified. Otherwise do nothing and return 204.
- Optionally drop the endpoint and let a scheduled job remove unverified records older than N hours.

Done when: no app call can delete a verified or pre-existing user.

Step 3 + API 7 · Upload documents to SharePoint

Why it moves

- The app calls the Power Automate flow with `EnableAuthorization: false` and any `attachmentid`: anyone can overwrite another user's photos or ID.
- Upload results are ignored; Under Review is set even if a file failed.
- Required-file rules live only on the phone.

Backend must (`/registration/documents` , multipart)

- Require `com_mobileverified` = true for the token's record (`MOBILE_NOT_VERIFIED`).
- Owner rules: `selfie` and `idFront` required; `idBack` optional (`DOCUMENTS_MISSING`). Accept JPEG/PNG only, set a size limit.
- Call the same flow server-side per file with the same body (`attachmentid` = the token's `com_userid` , `entityname`: "com_users" , field pairs as in API 7). Check every result; any failure → `UPLOAD_FAILED` and no status change.
- Optional: re-run a face check server-side; the phone check stays for instant feedback.

Done when: a failed upload leaves the record not Under Review; one user can't write to another user's fields.

API 8 · Set status to Under Review

Why it moves

- The app sets the status, and today it changes nothing because creation already set it.

Backend must

- Set `statecode: 0` , `statuscode: 1` only at the end of `/registration/documents` , after all required files are stored and the mobile is verified.

Done when: a record is Under Review if and only if it has a verified mobile and the required files.

Side entry · Resume from login

Why it moves

- Login filters on `com_password eq '{password}'` , returns the row with `com_password` , and re-compares on the phone.
- The phone decides where to resume from `statuscode` and the photo path, and an owner skips OTP even when the mobile was never verified.

Backend must

- Check the password server-side (hashed) and never return it.

- For an unfinished sign-up, return a `registrationToken` and `nextStep: verify-otp` if `com_mobileverified` is false, else `upload-documents`. Rejected with `com_allowresubmission = true` → `rejected-can-resubmit` + token.

Done when: an owner who never verified the mobile is always sent to OTP first.

Reference

CRM fields used by owner registration

Entity	Field	Meaning
com_users	com_userid	Record id
com_users	com_name, com_email, com_birthdate	Profile (name stored lower-cased today)
com_users	com_mobilenumber	+{dial}{mobile}
com_users	com_nationalid / com_passportnumber	One of the two, by the ID toggle
com_users	com_password	Plain text today
com_users	com_registeredas	Role: 1 Owner, 2 Family member, 3 Tenant, 4 Household, 5 Tenant family member, 6 Ref, 7 Ref owner
com_users	com_requestotp	Set true → CRM plugin generates <code>com_otp</code> , sets <code>com_otpepirydate</code> , sends SMS
com_users	com_otp, com_otpepirydate	Current OTP and its expiry (expiry ~6 h off today)
com_users	com_mobileverified	Mobile confirmed by OTP
com_users	com_appnotificationtoken	Firebase push token
com_users	com_profilepicturefilename / ...filepath	Selfie in SharePoint; path filled = "registration complete" today
com_users	com_nationalidfrontname / ...frontpath, com_nationalidbackname / ...backpath	ID images in SharePoint

Entity	Field	Meaning
com_users	com_allowresubmission	Rejected user may resubmit documents
accounts	new_mobilenumber , new_cbrnumber , blser_passportno	PHD customer match keys

Status values on com_users

statecode	statuscode	Meaning
0	1	Under Review (also set at creation today)
0	181410000	Approved by PHD
0	181410001	Rejected
1	2	Deactivated

Error codes and today's app messages

Code	HTTP	App shows today
VALIDATION_FAILED	400	Field errors (field required, not valid, enter a valid email)
USER_EXISTS	409	"Mobile number already exist"
INCOMPLETE_PROFILE	200 (info, flow continues)	"User exists but with incomplete profile, Please complete it."
NOT_PHD_CUSTOMER	403	"User dose not have an account in Palm Hills"
OTP_INVALID	400	"OTP not correct"
OTP_EXPIRED	400	New: verify button is disabled today
OTP_ATTEMPTS_EXCEEDED	429	New
OTP_RESEND_TOO_SOON	429	New: hidden behind the 60 s timer today
MOBILE_NOT_VERIFIED	403	New
DOCUMENTS_MISSING	400	"Please pick required attached"

Code	HTTP	App shows today
UPLOAD_FAILED	502	"Error while sending file"
REGISTRATION_TOKEN_INVALID	401	New: restart registration

Open questions to confirm before building

- ☐ Unfinished user restarts with new name/email/password: overwrite the old record, or keep it? (Today the new values are dropped.)
- ☐ Password hashing changes login for every user type. Migrate all users at once, or hash on next login?
- ☐ Which status means "started, not submitted"? Add a new `statuscode` value, or derive it from `com_mobileverified` + photo path?
- ☐ Is there a lookup field on `com_users` to link the matched `accounts` record?
- ☐ In what format is `accounts.new_mobilenummer` stored (with `+20`, with leading `0`)?
- ☐ OTP rules: expiry length, max attempts, resend cooldown, max sends per hour.
- ☐ Fix the `com_otpexpirydate` 6-hour offset in the CRM plugin, or correct it in the API?
- ☐ Keep `/registration/cancel`, or expire unverified records with a scheduled job?